

03

Workflow Automation with AI

Integrating QA and API testing automation

Four places a generated suite goes wrong that your existing habits do not cover — because the problem is new.

The programme

Four sessions, one thread: from one working agent to a governed fleet.

DAY 1

Fundamentals

What an agent is, how it runs, where it belongs.

DAY 2

Processes

One judge became a pipeline that runs unattended.

DAY 3

Testing

Agents write the tests.
Four places it goes wrong.

DAY 4

Ecosystem

Many agents, one owner, run records, and the bill.

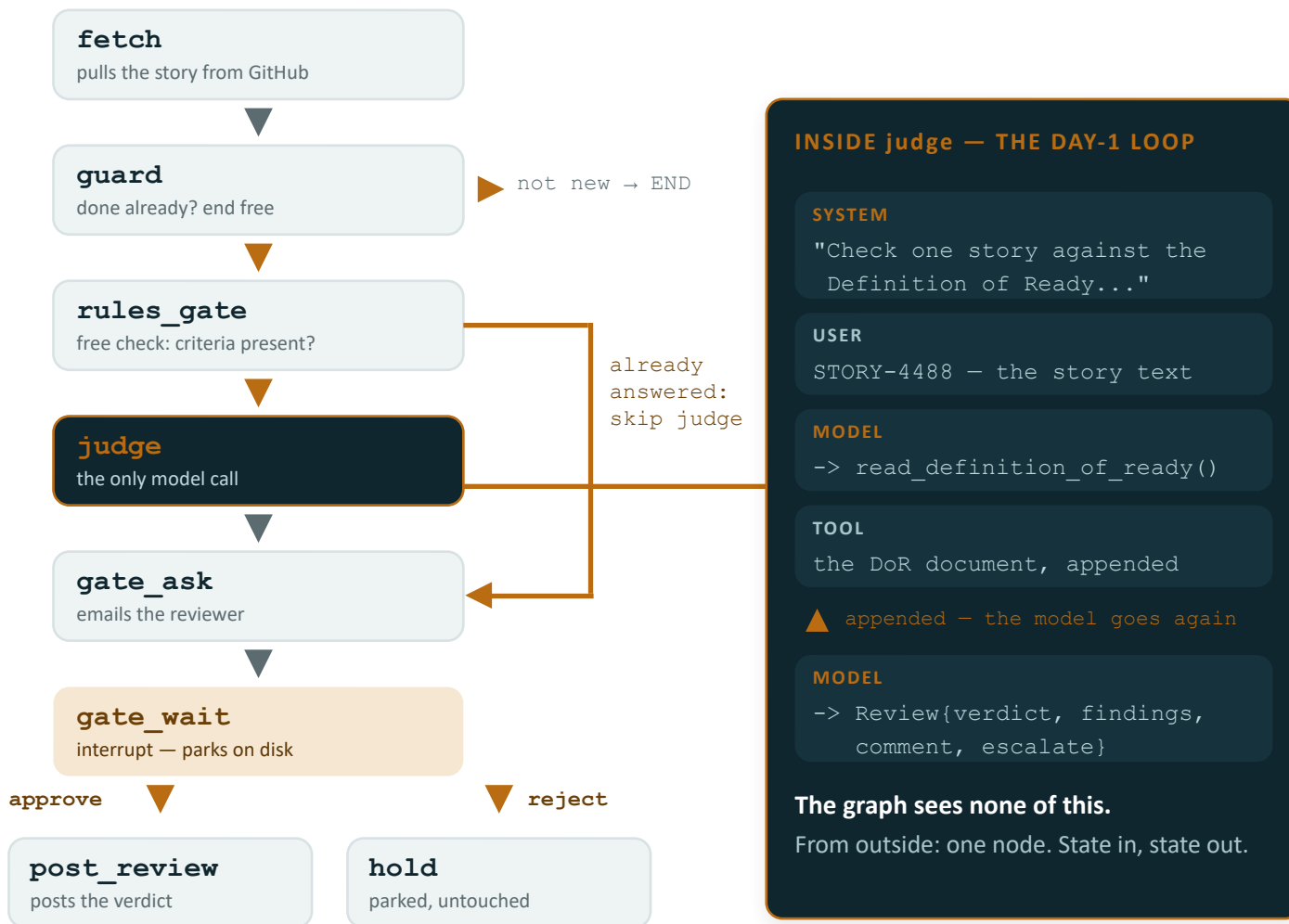
The API we test today is the one day two's stories were written about.

Same note cap, same closed-case rule, same rate limit. Yesterday this room refined those stories. Today we find out whether the code kept its promises.

Today: what does an agent-written suite actually get wrong, and how do you catch it without reading it?

Review — what we built yesterday

One story, fetched to posted — and the judge opened up to show the loop inside.



END — either way, the full run record is on disk.

THE VOCABULARY, AS WIRED

node

A plain function. State in, state out. fetch and hold are the same shape.

edge

Always-next. After fetch, guard. No decision involved.

conditional edge

A router reads the state and picks the next node: approve → post_review, reject → hold.

thread + loop

Inside the judge: the message list grows, the model re-reads it, and the loop ends on a valid Review.

interrupt

gate_wait stops the run mid-flight; its whole position is written to disk.

checkpointer

SqliteSaver. Why a reply from a phone resumes the run instead of restarting it.

amber arrow = a router deciding · grey arrow = always-next

Today — agents, then knowledge

Two halves, one machine.

FIRST · AGENTS THAT TEST

design_tests

an agent writes the suite from agreed criteria

run_suite

plain python fires it at a live, broken API

trriage_failures

an agent explains every red line — wrong twice

you watch each one alone, then the wiring

THEN · RAG — GIVE AGENTS KNOWLEDGE

ingest

nine files chunked, labelled, embedded

search

by meaning, by symbol, by label

ask

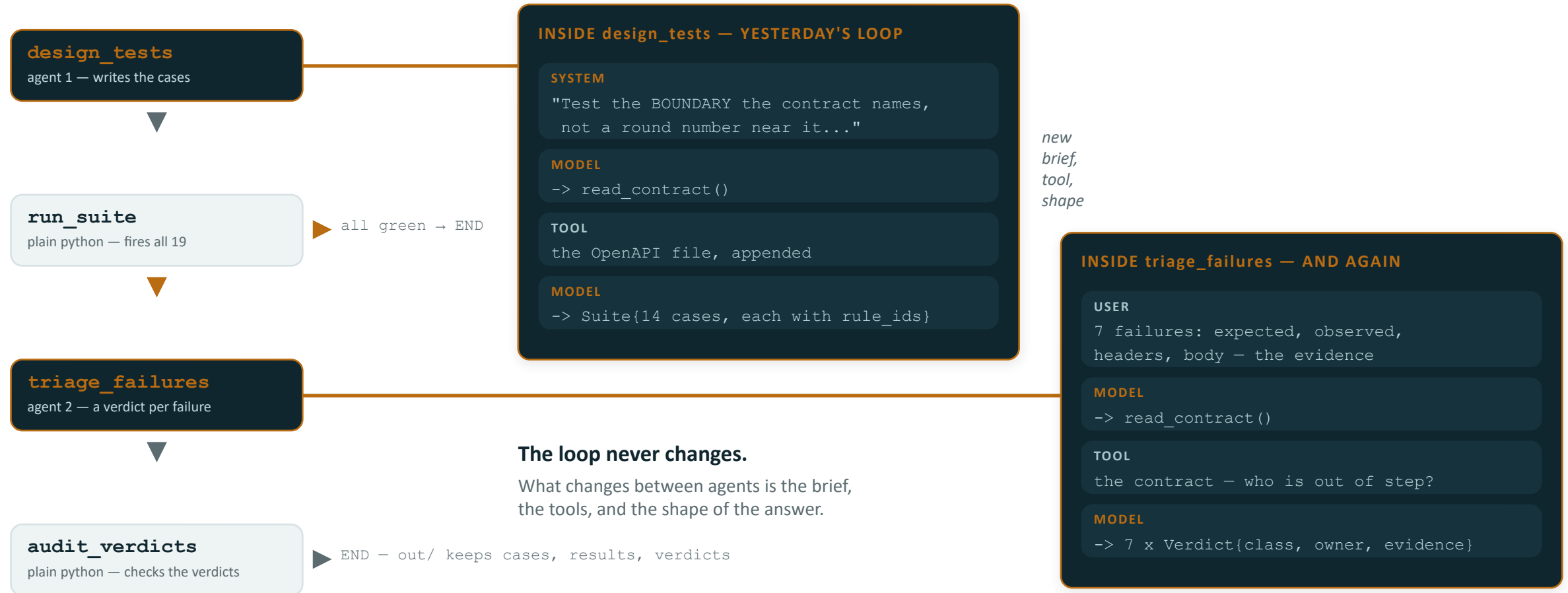
an agent that answers — and cites its sources

built live, from files to grounded answers

One loop all day. What changes is what you feed it — and what you check when it answers.

What we build today

Two agent nodes, two plain-Python nodes, one graph — and the same loop inside both agents.

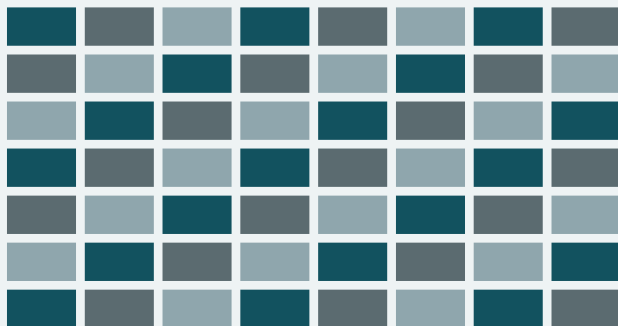


A green run ends at the router and costs nothing. Failures pay for exactly one loop: triage.

The thread is small. The estate is huge.

Every task needs a tiny slice of everything your organisation knows.

THE ESTATE

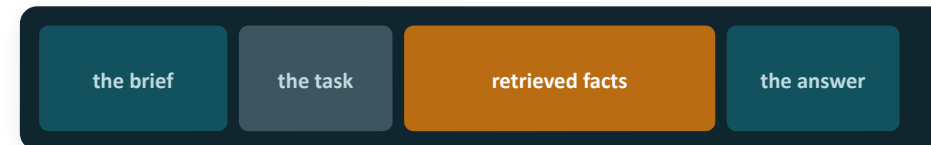


specs · code · incidents · ADRs · tickets



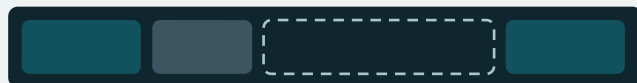
"which slice does
THIS task need?"

ONE RUN'S THREAD — THE WHOLE WINDOW



the only slot you control
a few pages — that is the entire window

SEND NOTHING



slot empty —
the model guesses

fluent, plausible, anchored to nothing

SEND EVERYTHING



does not fit

cost, latency — and the model loses the plot mid-document

So every serious agent has a selection step. Retrieve first. Reason second.

Two kinds of knowledge

One kind you index tonight. The other you must ask for, live.

AGES WELL — INDEX IT

true tonight, still true next month

specs · contracts

ADRs · decisions

past incidents

standards

runbooks

old test cases

search the index — RAG

LIVE THIS MINUTE — ASK FOR IT

stale before you finish indexing it

build status

logs, right now

the code as of HEAD

ticket states

test results

deployments

call a tool — live state

THE ENGINEERING AGENT

It draws on both. This section builds the left-hand path; the right-hand path is the tools you built all week.

The selection step, in one run

Captured from a real run of the agent you will watch shortly.

THE QUESTION

"What should tests cover for password reset?"

THE AGENT'S OWN SEARCHES — NOBODY SCRIPTED THESE

```
-> search_knowledge("QA checklist password reset flow tests security requirements...")
-> search_knowledge("...token email link expiry single use rate limiting enumeration...")
-> search_knowledge("...error handling lockout captcha abuse prevention")
-> search_knowledge("...Retry-After 429 case insensitive email reset request")
```

THE ANSWER, GROUNDED

45-minute token, single use
three resets a day — the fourth is 429
SSO accounts: 302, no reset e-mail
uppercase e-mails: the INC-2214 trap

WHAT IT STOOD ON

```
auth-service-spec::password-reset-token-rules
auth-service-spec::sso-managed-accounts
INC-2214::what-happened

— and it named what the knowledge does not cover.
```

RAG — fetch the relevant knowledge, put it on the thread, then reason. The next slides build exactly this.

Build the index — live

python ingest.py — nine files to a searchable index, narrated.

knowledge/

9 hand-written docs: specs, incidents, runbook, ADR, standards

PARSE

front-matter labels + markdown out of every file

CHUNK

17 chunks — one ## section each, cut on meaning

LABEL

service · doc_type · version, onto every chunk

EMBED

17 × 3,072 numbers · 853 tokens · \$0.0001

index.json

vector + text + labels, stored together — 1 MB

ONE RECORD, AS STORED

```
{ "id": "auth-service-spec::  
  login-throttling",  
  "service": "auth",  
  "doc_type": "spec",  "version": "2.3",  
  "text": "## Login throttling  
    ...throttled after five failed  
    attempts... ERR_AUTH_1042...",  
  "vector": [-0.0442, 0.0056,  
             -0.0140, ... x 3,072] }
```

The whole trick in one record: what it says, whose it is, where it sits on the map.

Built once, in five seconds, for a hundredth of a cent. Rebuilt only when the knowledge changes.

Chunking — cut where the meaning cuts

The chunk is what the model will eventually read.

THE TEST A CHUNK MUST PASS

Handed to you alone — no filename, no neighbours — could you still act on it?

THE DOCUMENT — auth-service-spec.md

```
## Password reset — token rules  
    valid for 45 minutes... single-use...
```

```
## Password reset — SSO-managed accounts  
    ...answers 302, no reset e-mail...
```

```
## Login throttling  
    ...five failed attempts... 429...
```

every ## heading is a cut — a rule, whole

ONE CHUNK, AS STORED

```
## Login throttling  
Authentication requests are throttled  
after five failed attempts within ten  
minutes... HTTP 429 with error code  
ERR_AUTH_1042... Retry-After header...  
clears 15 minutes after the last failure.
```

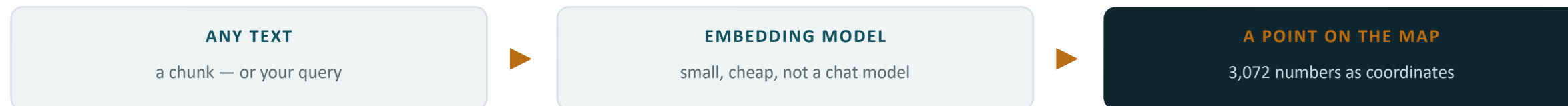
passes the test: actionable with no neighbours

Cut mid-class and the vector blurs five topics. Cut a bare line and the fact loses its owner.

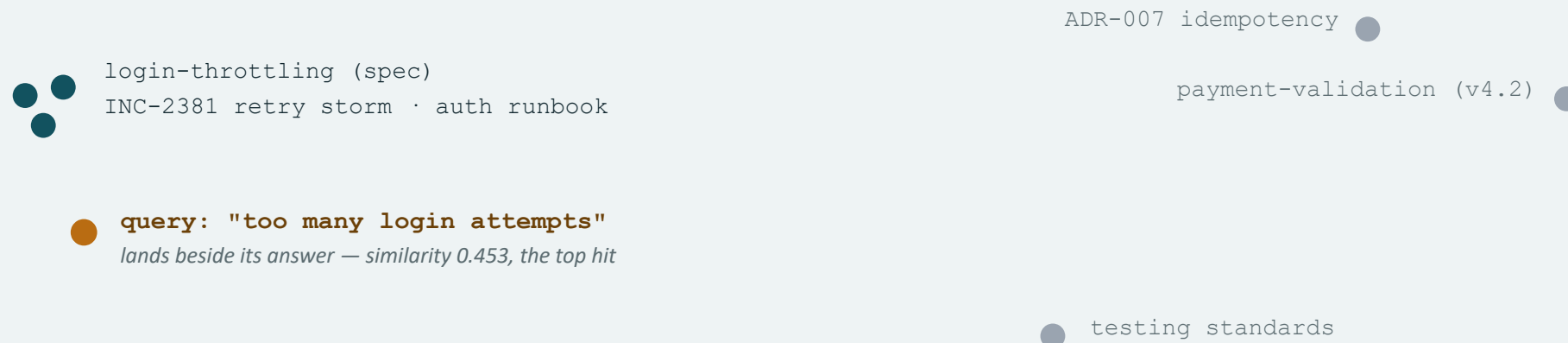
Engineering content has natural boundaries — headings, functions, endpoints, incidents. Never five hundred characters.

Embeddings — the meaning-map

Drawn from the real vectors in index.json.



THE MAP — POSITIONS PROJECTED FROM THE REAL VECTORS



One rule governs the map: similar meaning lands nearby — spelling is irrelevant. Auth talk clusters left; checkout sits far away.

Chunks are embedded once, at indexing. A query is embedded when asked — same map, nearest neighbours win.

Hybrid search

Three kinds of looking — all three ran a minute ago.

SEMANTIC

"too many login attempts"

→ **login-throttling** · 0.453

wins on paraphrase

KEYWORD / SYMBOL

ERR_AUTH_1042

→ **EXACT SYMBOL MATCH, boosted**

wins on identifiers

METADATA FILTER

--service checkout --version 4.2

→ **2 of 17 chunks in scope**

wins by scoping first

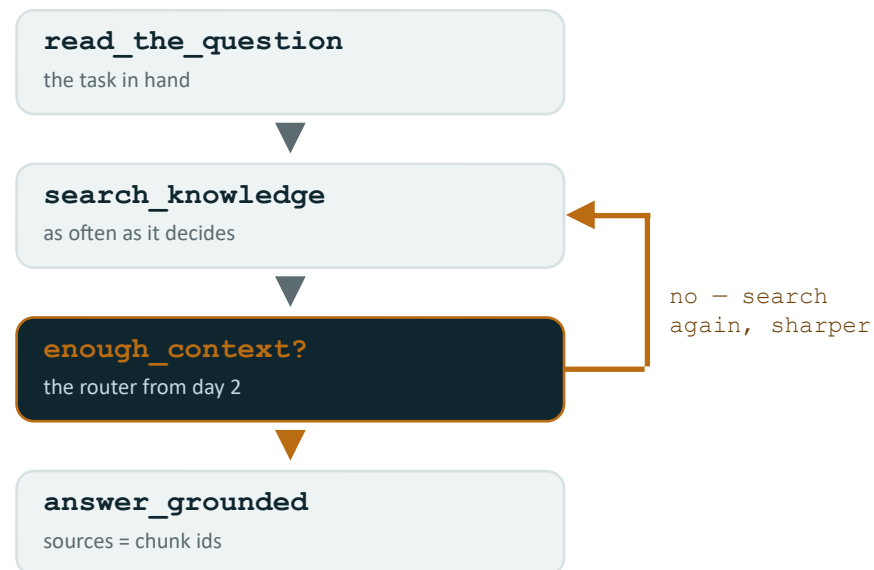
An error code is the counter-example that keeps you honest:

for ERR_AUTH_1042, "similar meaning" is precisely what nobody wants — a related code is a different error.

Serious engineering retrieval runs all three in one query and merges the results. Every time.

RAG inside the agent

One tool — `search_knowledge` — and the loop you already know.



This morning's loop. The only new thing is what the tool returns.

THE RUN YOU JUST WATCHED

4 searches it planned itself
5 facts in the answer — every one retrieved:
45-min single-use token · 3/day then 429
SSO → 302 · the INC-2214 uppercase trap
3 chunk ids cited as sources

And the sixth point: "the knowledge does not cover X" — a gap named, not filled.

AGENT QUALITY = REASONING QUALITY × CONTEXT QUALITY

The first factor you buy. The second — chunks, labels, hybrid search — you just built.

~\$0.01 a question, and every claim traceable to a chunk. That is retrieval-augmented generation, in an agent.

Should everything become RAG?

No — and you already know the rule.

KNOWLEDGE — AGES WELL → RAG

architecture documentation
existing test cases
coding guidelines
historical incidents

STATE & ACTION — LIVE → TOOL

current build status · live logs
execute the test suite
create a Jira defect
modify source code

source code sits between: retrieve to understand, tool to touch.

RAG

= KNOW

TOOLS

= DO · GET LIVE STATE

LLM

= THINK

LANGGRAPH

= ORCHESTRATE

Knowledge ages well — retrieve it. State changes by the minute — call a tool for it.

Three things to carry

1

Ground the agent in agreed criteria.

The contract decided everything today: what was worth testing, and which side of a failure was wrong. Take the knowledge away and the model does not go quiet — it produces fluent tests that prove nothing.

2

The loop never changes.

The designer, triage, and the ask agent were the same machine: a model, its tools, a brief, and the shape of the answer. Those four parts are what you design — the loop comes for free.

3

Retrieve first. Reason second.

And context quality is yours: the chunks, the labels, the hybrid search. Agent quality = reasoning quality × context quality — you control the second factor.

Questions — and same time for day four.